



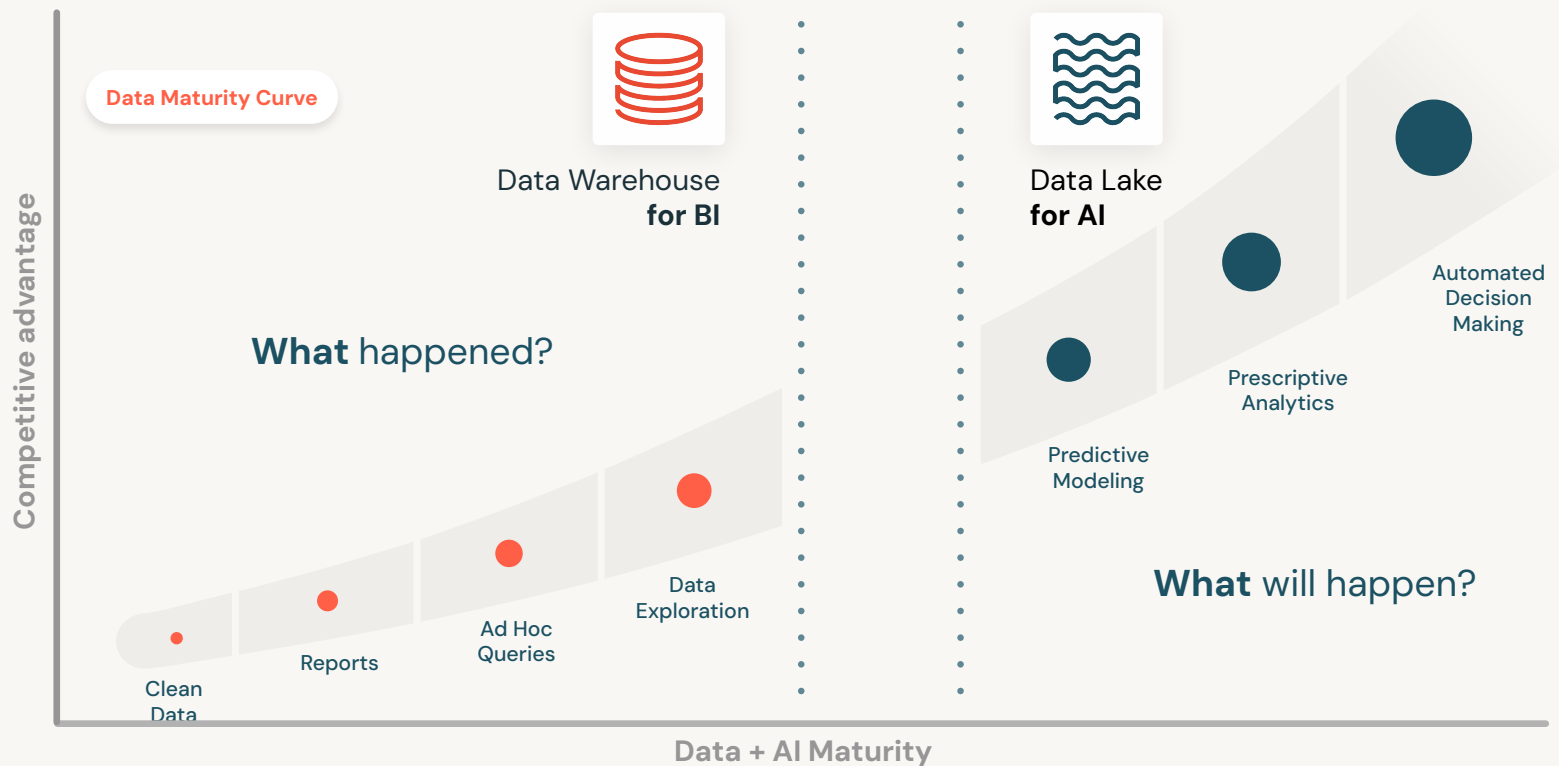
Dimensional & Data Vault Modeling for Lakehouse

How to get the best star schema performance in Lakehouse



Legacy approach

Two disparate, incompatible data platforms

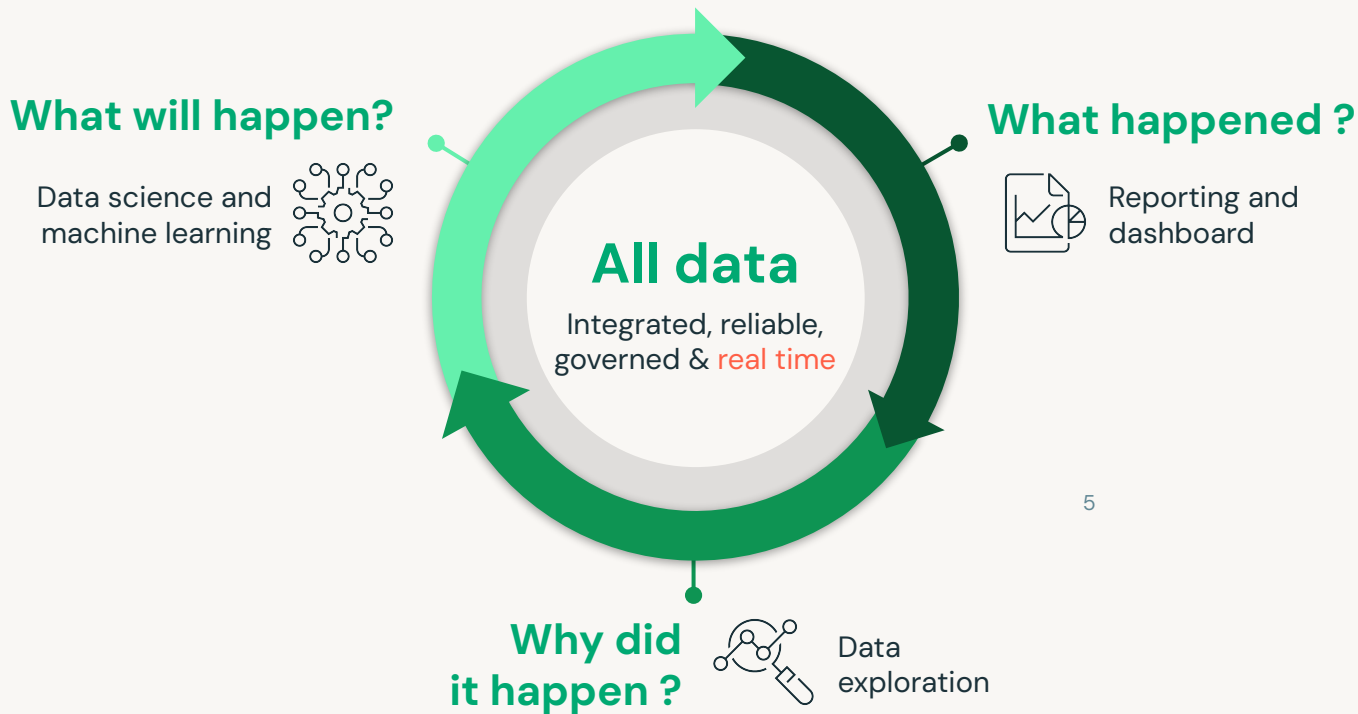


Dimension Modeling



Needs of data driven enterprise

Collaborative platform with consistent data and closed loop analytics

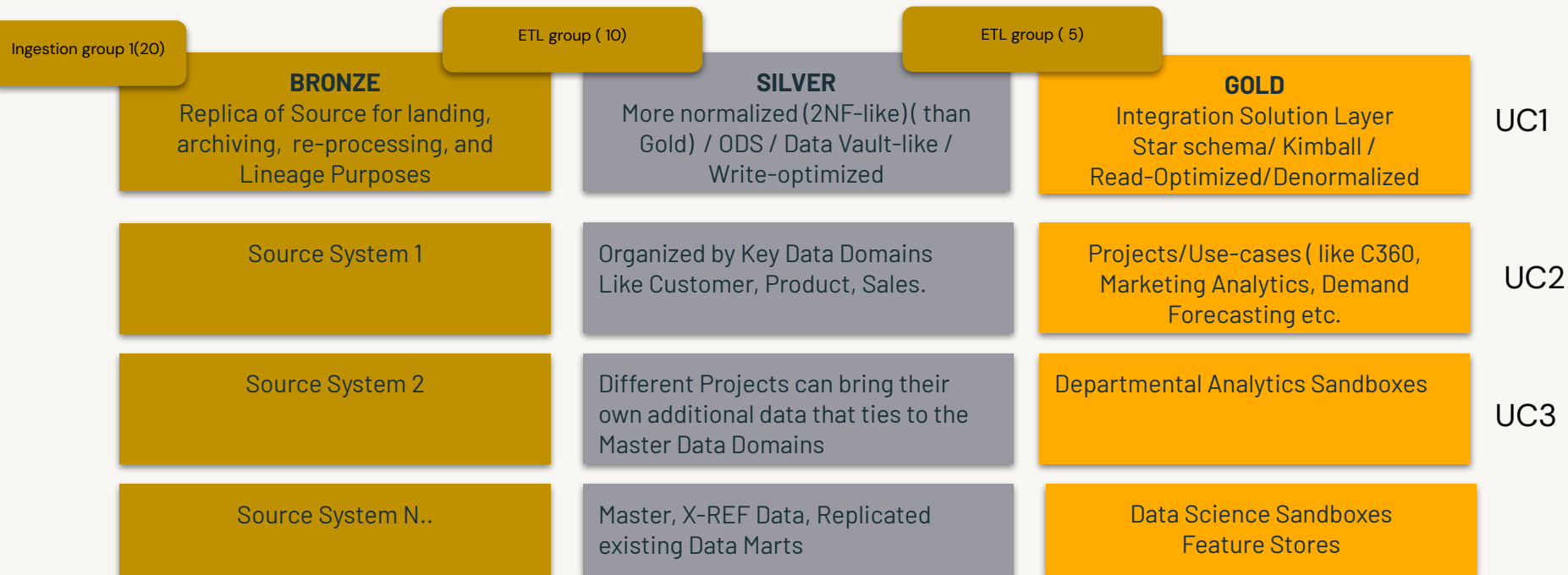


5

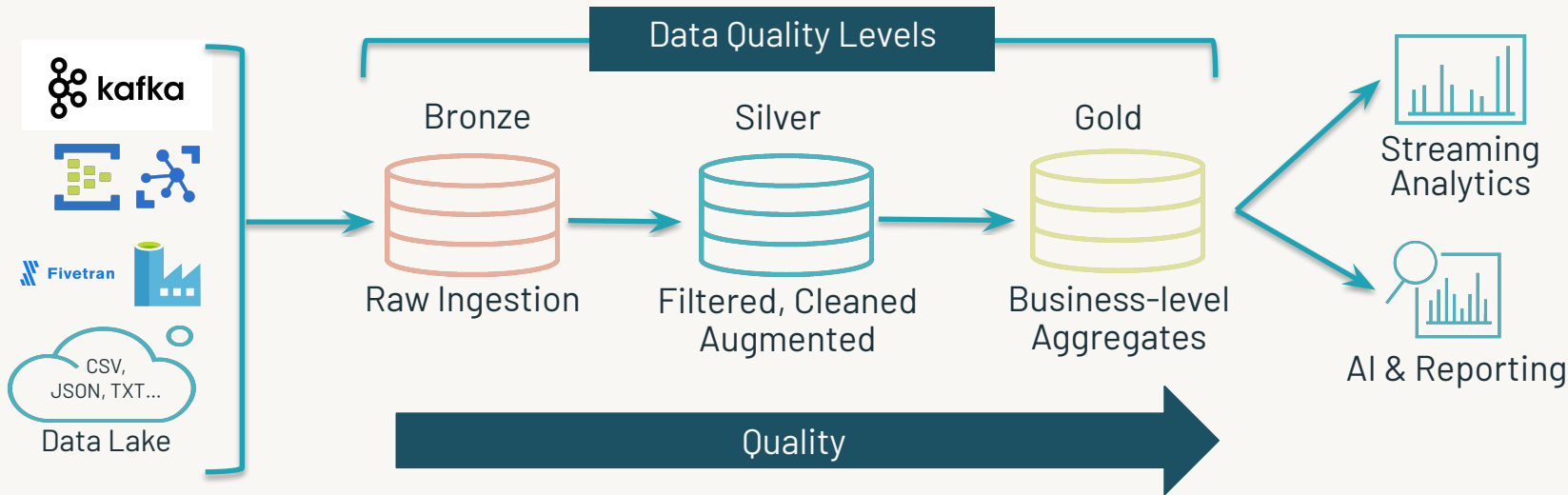


Lakehouse Medallion Zones

General Guidelines



Lakehouse Medallion Architecture



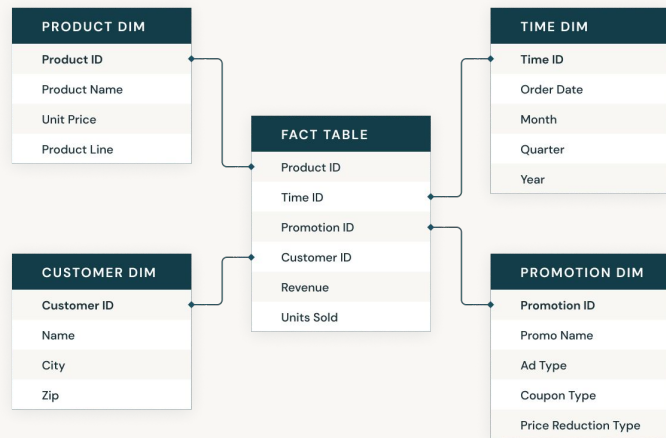
Delta Lake allows you to *incrementally* improve the quality of your data until it is ready for consumption.

Dimensional Modeling

Kimball Approach

- Denormalize dimension and fact tables
- Use conformed dimensions
- Create and use surrogate keys for joins
- Use slowly changing dimensions as necessary
- Aggregate tables for scalability using DLT (Materialized Views) or in BI semantic layer tools

Star schema



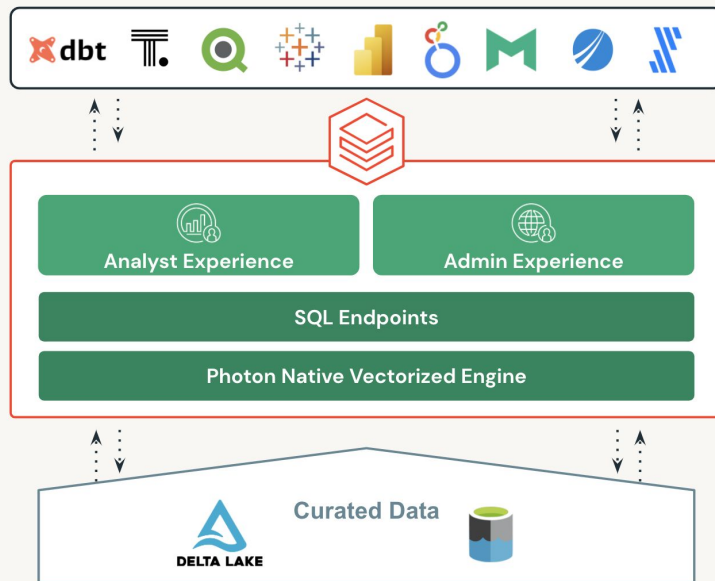
BUSINESS PROCESSES	SHARED DIMENSIONS									
	Date	Customer	Product	Vendor	Promotion	Reseller	Sales Territory	Employee	Account	Organisation
Internet Sales	✓	✓	✓	✓	✓	✓	✓			
Reseller Sales	✓		✓		✓	✓	✓	✓		
General Ledger	✓								✓	✓
Sales Plan	✓		✓				✓			
Inventory	✓		✓	✓					✓	
Customer Surveys	✓	✓								
Customer Service Calls	✓	✓	✓					✓		



Dimensional Modeling

Use Delta Tables & Databricks SQL (Photon)

- Benefits of Delta Lake:
 - ACID Transactions & Schema Enforcement
 - Batch & Streaming consolidation
 - Insert/Update/Delete/Merge
 - Data Skipping, Indexing, & Caching
 - Time Travel
 - Change Data Feed
- Column Stats on first 32 columns by default – configurable
 - This enables data skipping/dynamic file pruning
- Use Databricks SQL (with Photon) for the best BI performance



Dimensional Modeling

Optimize file size

- Set ideal data file size for data skipping and BI queries. Options:

- a. Set the file size for tables

- general recommendation range is 32-128MB

```
alter table database_name.table_name set  
tblproperties('delta.targetFileSize' = 32mb);
```

- b. Set the file size using Autotune for workload

- c. Use the file size Autotune (default)

- Sizes for dimensions and facts could be different
- Don't over-partition tables
- Use Auto Optimized during ETL
- Run Optimize Jobs daily

Table size	Target file size	Approximate number of files in table
10GB	256MB	40
1TB	256MB	4096
2.56TB	256MB	10240
3TB	307MB	12108
5TB	512MB	17339
7TB	716MB	20784
10TB	1GB	24437
20TB	1GB	34437
50TB	1GB	64437
100TB	1GB	114437

Autotune default settings

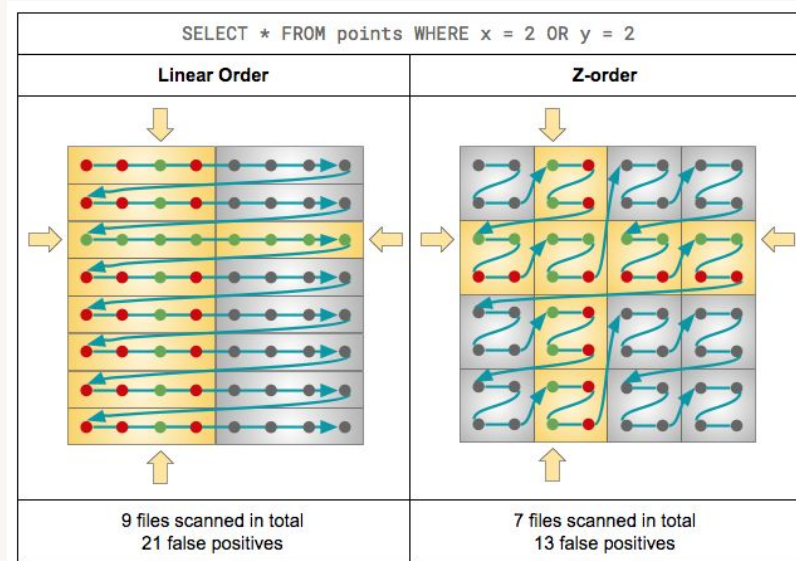


Dimensional Modeling

Z-Order Facts

- Limit columns in Z-Order to the best 1-4 columns
- Dimensions FKs that are too large for broadcast joins
 - Or the most frequently used FKs
- Consider partitioning to further improve file skipping – large scale

OPTIMIZE MY_FACT_TABLE ZORDER
BY (LARGEST_DIM_FK,
COMMONLY_USED_DIM_FK, ...)

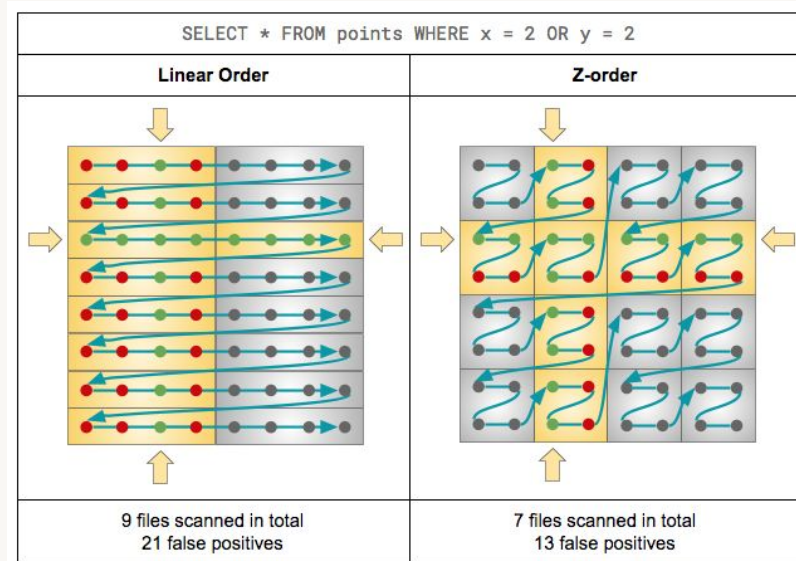


Dimensional Modeling

Z-Order Dimensions

- Limit columns in Z-Order to the best 1-4 columns
- Dimensions SK, common filter columns
- Consider partitioning to further improve file skipping – large scale

OPTIMIZE MY_DIM ZORDER BY
(MY_DIM_PK, LIKELY_FIELD_1,
LIKELY_FIELD_2)

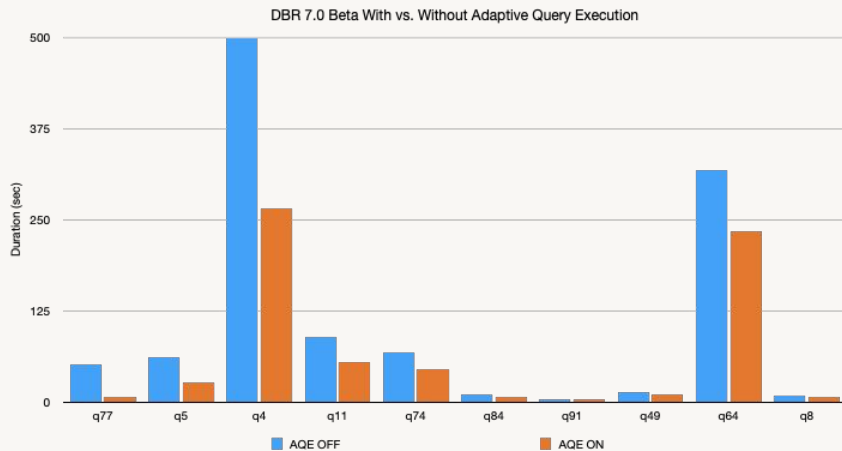


Dimensional Modeling

Analyze Tables

- Spark 3.0+, AQE provides accelerated performance for dimensional models
- Run Analyze Jobs daily to create table statistics
- Stats are used by AQE to make better plan decisions

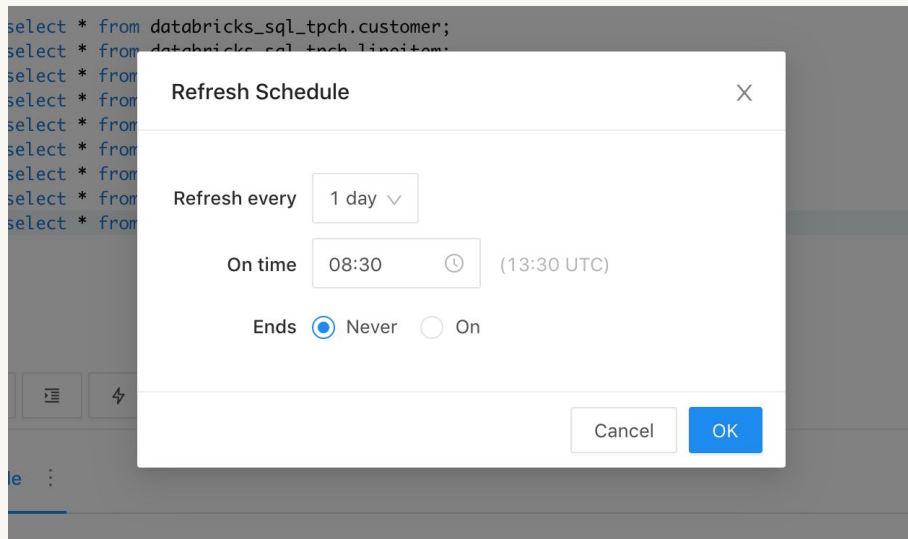
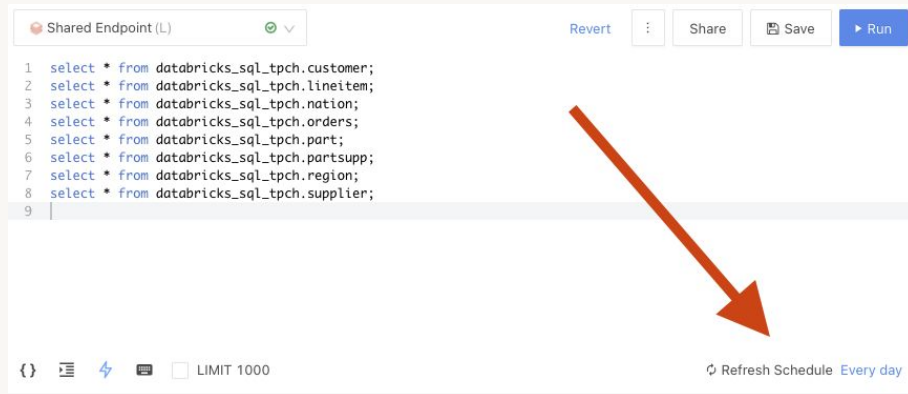
ANALYZE TABLE MY_DIM COMPUTE
STATISTICS FOR ALL COLUMNS



Dimensional Modeling

CACHE Tables

- Databricks SQL has a great caching engine
 - Query Results cache
 - SSD cache
- Can create a refresh schedule to cache the most common tables or queries
- This can be done directly in Databricks SQL



Generating Identity Columns

Old options for generating identity columns (surrogate keys)

monotonically_increasing_id – unique for the process, not for the table.

row_number() Rank OVER – requires a full table scan, shuffle, and sort.

zipWithIndex() – RDD function that is serialized in the driver

zipWithUniqueIndex() – an RDD function that runs in parallel

Row Hash with hash() – hash only the natural key

Row Hash with md5() – hash only the natural key

All of the old techniques had their own challenges and considerations and all are suboptimal!

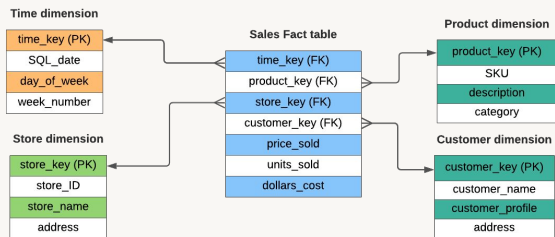


Support for Identity Columns, Primary + Foreign Key Constraints

DBR 10.4

IDENTITY COLUMNS

- Define **IDENTITY** column on a table
- Delta can **automatically** generate **unique integer values** when new rows are added to the table with IDENTITY columns
- Users can also explicitly insert values for the IDENTITY columns



DBR 11.1

PRIMARY + FOREIGN KEY DECLARATIONS

- Declare unenforced Primary and Foreign keys with **ALTER TABLE**
- Visible in **INFORMATION_SCHEMA** and DESCRIBE TABLE
- Allow end users to understand **relationships** between tables

GOAL: Enable **data quality** and **easy table relationship discovery** for tools and users that are not familiar with the data model.



Generating Identity Columns

Delta Lake Identity Support Starting in DBR 10.4+

```
CREATE TABLE IF NOT EXISTS dim_loan
(
  Loan_sk BIGINT GENERATED ALWAYS AS IDENTITY,
  Loan_id BIGINT,
  .....
)
USING DELTA
LOCATION 'abfs://<container>@<storage account>'
```

Options

```
ALWAYS | BY DEFAULT
START WITH start
INCREMENT BY step
```

- Generated columns only work when pre-creating table
- Always option doesn't allow column override
- By Default option does allow column override but *doesn't enforce duplicates*
- Start With option allows you to start anywhere
- Increment option allows you to set the increment
- Modify settings with ALTER



Primary Key/Foreign Key

Delta Lake PK/FK Support Starting in DBR 11.1 with UC

```
CREATE TABLE IF NOT EXISTS dim_loan
(
  Loan_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
  KEY,
  Loan_nk BIGINT,
  .....
)
USING DELTA
LOCATION 'abfs://<container>@<storage account>/'
```

```
CREATE TABLE IF NOT EXISTS fact_loanpayments
(
  Fact_id BIGINT GENERATED ALWAYS AS IDENTITY,
  Loan_id BIGINT NOT NULL CONSTRAINT dim_loan_fk FOREIGN
  KEY REFERENCES dim_loan,
  .....
)
USING DELTA
LOCATION 'abfs://<container>@<storage account>/'
```

- PK/FK Constraints starting in DBR 11.1
- Requires Delta and Unity Catalog
- Relationships are unenforced
- Information Schema can be used by BI tools to understand relationships



Dimension Table Modeling

Designing your Dimension tables on Delta

```
CREATE TABLE IF NOT EXISTS dim_loan
(
  Loan_sk BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, --Identity Columns
  Loan_id BIGINT,
  Funded_amnt INT NOT NULL,
  Paid_amnt DOUBLE,
  Addr_state VARCHAR(100), --Support native database string types
  Type STRING,
  Load_date TIMESTAMP NOT NULL,
  LoanYear INT GENERATED ALWAYS AS ( YEAR(Load_date) ),
  LoanMonth INT GENERATED ALWAYS AS ( MONTH(Load_date) )
)
USING DELTA
PARTITIONED BY (LoanYear)
LOCATION 'abfss://<container>@<storage account>/'
TBLPROPERTIES (
  'delta.autoOptimize.optimizeWrite' = 'true',
  'delta.enableChangeDataFeed' = 'true');
ALTER TABLE dim_loans ADD CONSTRAINT dateWithinRange CHECK (timestamp > '1900-01-01');
```

- Use Delta
- Set Table Properties
- Set Location
- Generate SKs
- Set Primary/ Foreign Keys
- Can use more native SQL data types like varchar()
- Create constraints as needed
 - Null and Not Null
 - Add Constraint
- Partition as needed
- Optimize and Analyze



Features to make DDLs/Data Modeling easier in Databricks

(E.g. Mat Views, DLT , Identity columns / PK/FK Generate SK's in code)

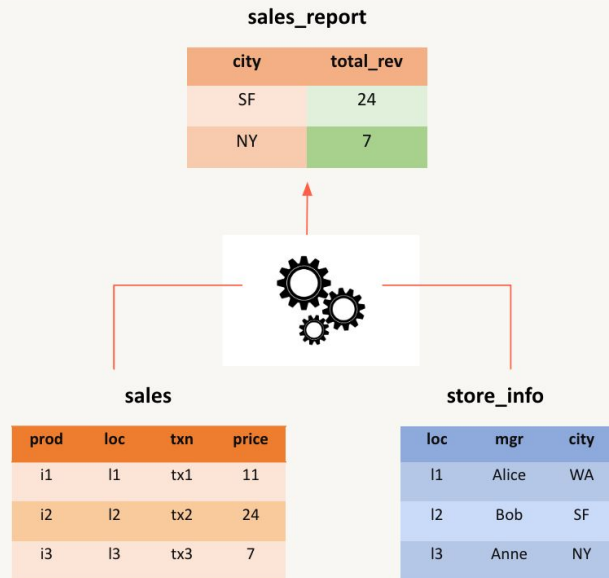


Materialized Views

Speed up queries with pre-computed results

Accelerate end-user queries and reduce infrastructure costs with efficient, incremental computation

- Accelerate **BI dashboards** and **ETL queries**
- **Streaming**: build MVs on top of live tables
- **Easy ELT**: Simplify reporting by cleaning, enriching, denormalizing the base tables
- **Data Sharing & Access Control**: control what info can be seen by internal and external users and organizations



Materialized Views

Powered by Delta Live Tables

```
(  
  CREATE MATERIALIZED VIEW [ IF NOT EXISTS ]  
  | CREATE OR REPLACE MATERIALIZED VIEW  
)  
[[<catalog>.<database>.<name>  
  SNAPSHOT  
  [COMMENT <comment text>]  
  AS <SQL query>
```

```
ALTER TABLE table1 SET TBLPROPERTIES  
(delta.enableChangeDataFeed = true)
```

```
CREATE MATERIALIZED VIEW mv1  
SNAPSHOT  
AS  
SELECT date AS date, sum(sales) AS  
sumOfSales  
FROM table1  
GROUP by date
```

```
REFRESH MATERIALIZED VIEW <mv_name>
```

```
ALTER MATERIALIZED VIEW <name>  
  
{ rename } TO <name>
```



SCD Type 2 Pattern



Slowly Changing Dimensions Type 2+

Code Patterns for Batch

- Works using SQL, Python, and Scala
- Can use Merge or use separate operations to update and insert rows
- Hash of “slowly changing” attributes can provide an easy approach for comparing values

```
MERGE INTO customers
USING (
  -- These rows will either UPDATE the current addresses of existing customers or INSERT the new add
  SELECT updates.customerId as mergeKey, updates.*
  FROM updates

  UNION ALL

  -- These rows will INSERT new addresses of existing customers
  -- Setting the mergeKey to NULL forces these rows to NOT MATCH and be INSERTed.
  SELECT NULL as mergeKey, updates.*
  FROM updates JOIN customers
  ON updates.customerid = customers.customerid
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customerid = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, endDate = staged_updates.effectiveDate -- Set current to false and er
WHEN NOT MATCHED THEN
  INSERT(customerid, address, current, effectiveDate, endDate)
  VALUES(staged_updates.customerid, staged_updates.address, true, staged_updates.effectiveDate, null)
```



Slowly Changing Dimensions Type 2+

Code Patterns for Streaming

- foreachBatch allows streaming pipelines to execute MERGE commands

```
# Function to upsert `microBatchOutputDF` into Delta table using MERGE
def upsertToDelta(microBatchOutputDF, batchId):
    # Set the dataframe to view name
    microBatchOutputDF.createOrReplaceTempView("updates")

    # =====
    # Supported in DBR 5.5 and above
    # =====

    # Use the view name to apply MERGE
    # NOTE: You have to use the SparkSession that has been used to define the `updates` dataframe
    microBatchOutputDF._jdf.sparkSession().sql("""
        MERGE INTO aggregates t
        USING updates s
        ON s.key = t.key
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)
```

```
# Start the query to continuously upsert into aggregates tables in update mode
aggregatesDF.writeStream \
    .format("delta") \
    .foreachBatch(upsertToDelta) \
    .outputMode("update") \
    .start() \
```



Stored Procedure Migration Patterns

Where do Stored Procedures sit in Lakehouse

- SPs to Workflows
- SPs to Notebook
- SPs to Table Valued UDFs



What is a Warehouse on a Lakehouse?

The best Warehouse is a Lakehouse

- Define once, secure, govern, and use everywhere
- Separation of Warehouse compute and BI compute
- Multiple Warehouse compute ETL engines
- Delta Tables as the warehouse storage
- Write ACID compliance across multiple compute engines



Liquid Recommendations

New Partitioning Strategy

- Still only for data over 1TB?
- When to use Liquid
-
-
-
-



UniForm Format

Unification of Delta, Iceberg, and Hudi

-
-
-
-
-
-

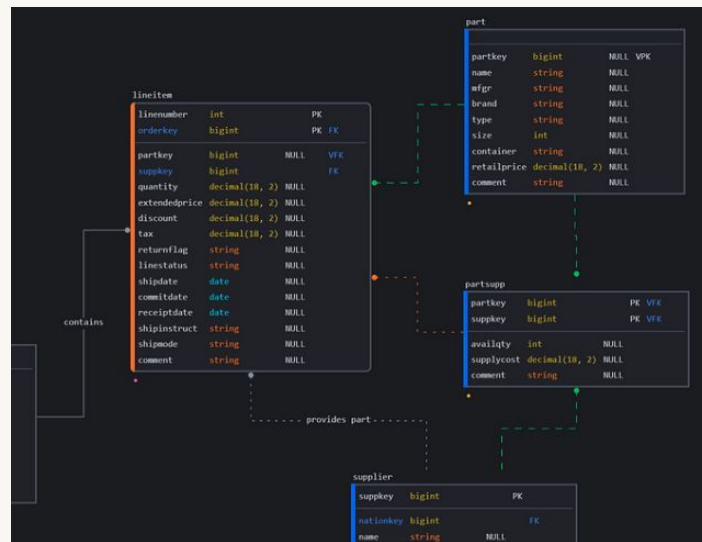


Data Modeling Tool Partners – SqlDBM, Quest Erwin



sqlDBM – our newest Data Modeling ISV partner

- Data Modeling Best practices on Databricks – sqlDBM
- Supports Reverse Engineering Data Models from Legacy EDWs
- Supports Data Model implementations in Databricks
- Integrates with Unity Catalog and supports PK/FK relationships



Quest and Databricks - Joint Value

Customer Challenges

- Replicating and consolidating legacy data models into the Delta Lake, Building data models for the Delta Lake
- Document, data & lineage across multiple platforms, manage compliance and access
- Build trust in data to make better business decisions

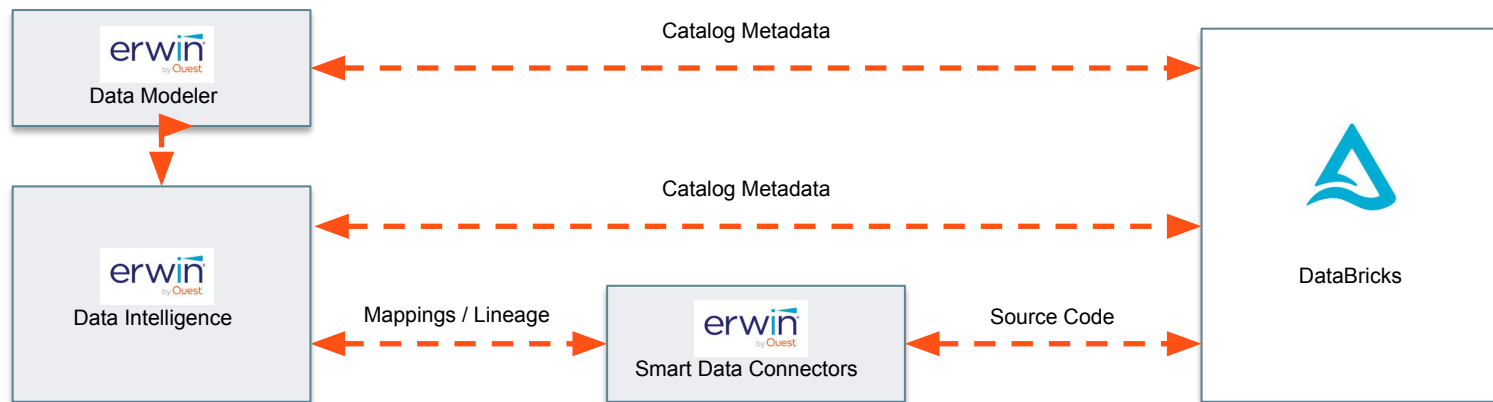
Solution

- erwin Data Modeler
- erwin Data Intelligence
- erwin Data Connectors

Benefits

- Save time, mitigate risk when migrating through well architected data structures
- Improved data accuracy/quality by replicating existing data models to Databricks
- Accelerate migration of data to the lakehouse
- Reduce compliance risk
- Provide greater visibility, impact analysis and more through automation
- Self-service data visibility for all business users
- Harvest metadata from about any data source

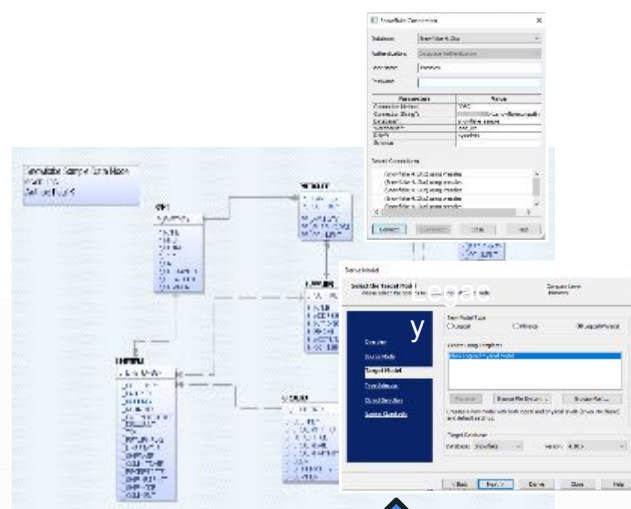
Joint Architecture - Quest/Databricks



Building Data Models

Reverse Engineering: Generate legacy data structure technology models

Native Modeling: Design / redesign target model structures and characteristics



Databricks SQL

Structured Semi-Structured Unstructured Streaming



- Harvest & Analyse Databricks metadata

- Structure and Govern assets using Data Literacy

DATA INTELLIGENCE SUITE

Microdata Manager

Dashboard

Explore

Sensitive Data

Metadata

3rd Party Flat Files

Accounting System

Market Research

Reports

Customer Order Entry

Customer S3

CRM Modules

EMP - CRM Model (x1,10)

EDW

EDW Test

enron Models

FACETS

Finance and Billing

HR Records

Legacy Test

Mailhead

ODS

PeopleSoft

Pubx

Reference

Sample Databases

Schema Cdl Layer

SQL Server Databases

Teradata

Statistics

Total Primary Key Columns

15%

Total Foreign Key Columns

40%

Tables With Expanded Logical N.

0%

Columns With Expanded Logical

0%

OS Score

0%

Date Dictionary

Environment Details

Extended Properties

Data Lineage

Impact on Source

Impact on Target

Metadata

Associations

Workflow Log

Documents

Data Quality

Configure Extended Properties

Scheduled Jobs

Create Database

#

Options

Table Name

Column Name

Logical Column Name

Column Comments

Column Definition

Sensitive Data Indicator (Y/N)

Sensitive Data Indicator (SDI) Classification

Sensitive Data Indicator (SDI) Description

Data Type

Length

Precision

Scale

Default Value

Nullable Flag

Logical Name

1

EMP

EMP_number

EMP_number

A unique number that identifi...

A unique number that identifi...

Restricted

Restricted

Restricted

varchar

20

20

N

EMP#

2

EMP

EMP_address

EMP_address

A unique number that identifi...

A unique number that identifi...

Restricted

Restricted

Restricted

varchar

20

20

N

EMP#

3

EMP

EMP_emp_name

EMP_emp_name

Employee's name

Employee's name

Restricted

Restricted

Restricted

varchar

20

20

Y

EMP#

4

EMP

EMP_address

EMP_address

First line of the employee's a...

First line of the employee's a...

Restricted

Restricted

Restricted

varchar

20

20

Y

EMP#

5

EMP

EMP_phone

EMP_phone

Employee's home phone nu...

Employee's home phone nu...

Restricted

Restricted

Restricted

integer

Y

EMP#

6

EMP

EMP_address_2

EMP_address_2

Second line about address ...

Second line about address ...

Restricted

Restricted

Restricted

varchar

20

20

Y

EMP#

7

EMP

EMP_ssn_number

EMP_ssn_number

Social Security Number

Social Security Number

Restricted

Restricted

Restricted

integer

Y

EMP#

8

EMP

EMP_date

EMP_date

Employee's date of hire at ...

Employee's date of hire at ...

Restricted

Restricted

Restricted

datetime

Y

EMP#

9

EMP

EMP_sal

EMP_sal

Annual salary paid to the em...

Annual salary paid to the e...

Restricted

Restricted

Restricted

integer

Y

EMP#

10

EMP

EMP_email

EMP_email

email

email

Restricted

Restricted

Restricted

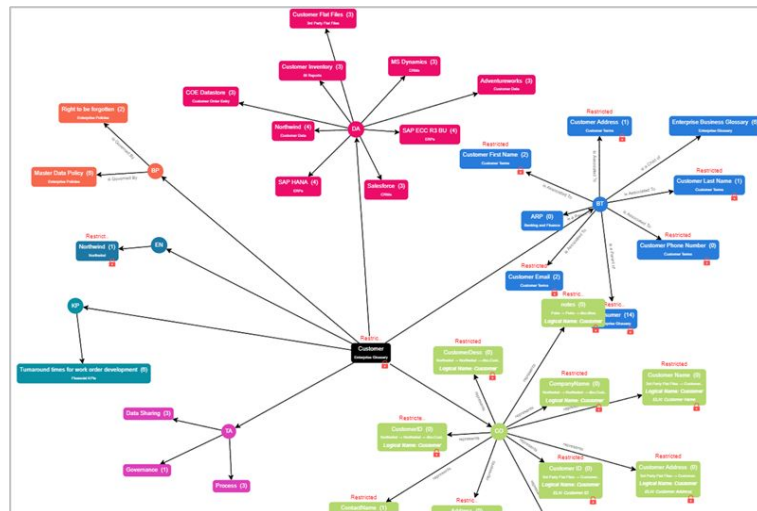
varchar

20

20

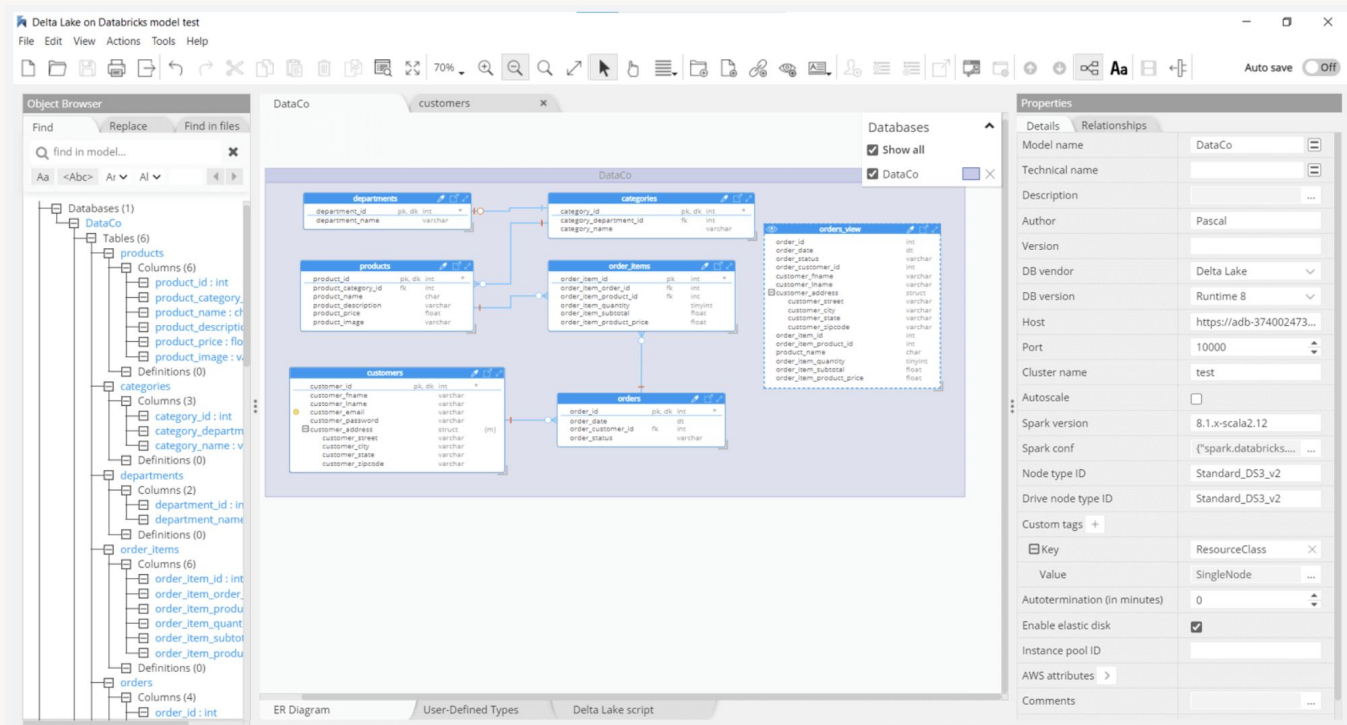
Y

EMP#



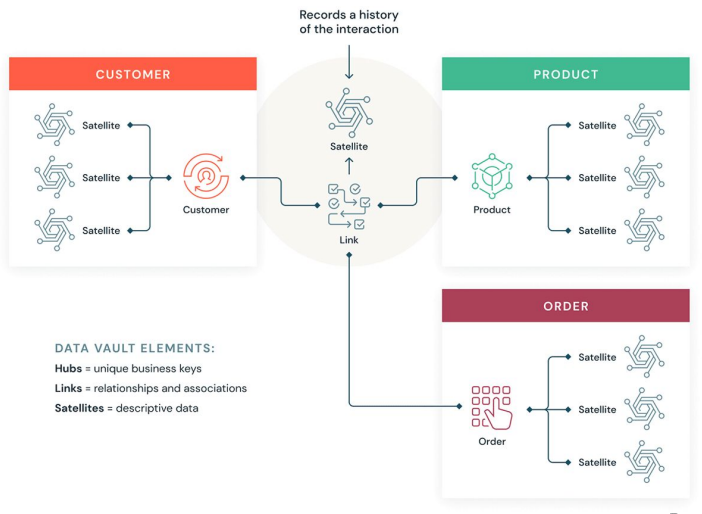
Hackolade Data Modeling tool..

Hackolade Data Modeling

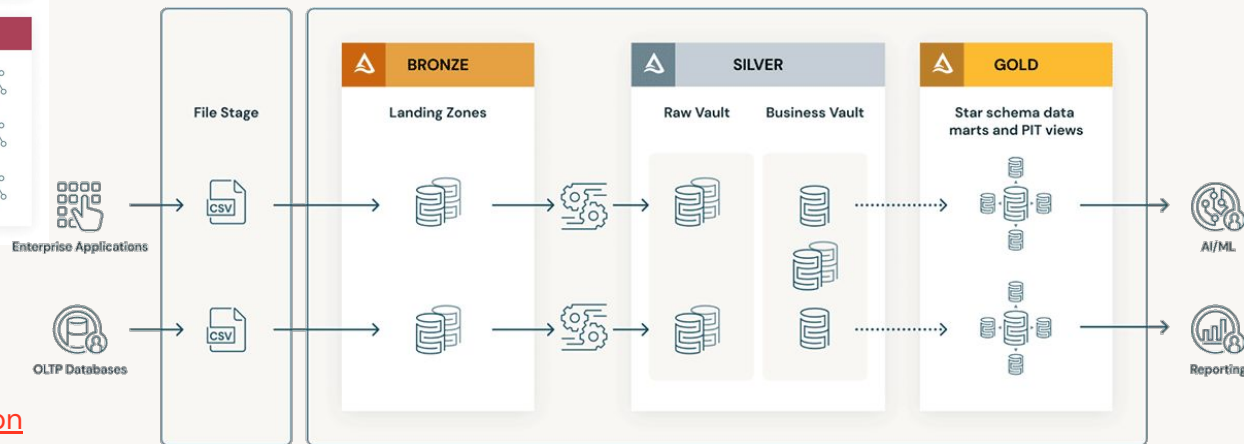


Data Vault Modeling

Recommended ONLY if your Organization has DV 2.0 Certified Data Modelers and you have a need to lift/shift/modernize existing Data Vault Models. (Else – requires learning a new DM paradigm with set rules – not recommended for green field implementations)



- **Landing Zone:** ~ Bronze (Avro, CSV, parquet, XML, JSON formats, it is converted to Delta-formatted tables in the staging zone..)
- **Raw Vault:** Data is modeled as Hubs, Links and Satellite tables
- **Business Vault:** ETL business rules, data quality rules, cleansing and conforming rules
- **Presentation/Gold:** Query-helper tables such as Point-in-Time (PIT) and Bridge tables are created on top of the business vault.



Partner: [VaultSpeed for DataVault automation](#)

Data Vault Modeling on Databricks

6 Basic Steps to Success

- Use Delta Formatted tables for Raw Vault, Business Vault and Gold layer tables.
- Do not over partition the tables –especially the smaller satellites tables.
- Use Z-order Indexing and Bloom filter indexing on Date columns, current flag columns and predicate columns that are typically filtered on to ensure best performance
- SDP (Spark Dec Pipelines) (Materialized Views) makes creating and managing PIT tables very easy.
- Smaller files = more file pruning = faster queries . (32-64MB vs. 1 GB.)
- Use latest version of Databricks Runtime (*it takes care of Adaptive Query Execution, auto-join hints*)



Partners – Vault Speed



VaultSpeed

DataVault automation partner

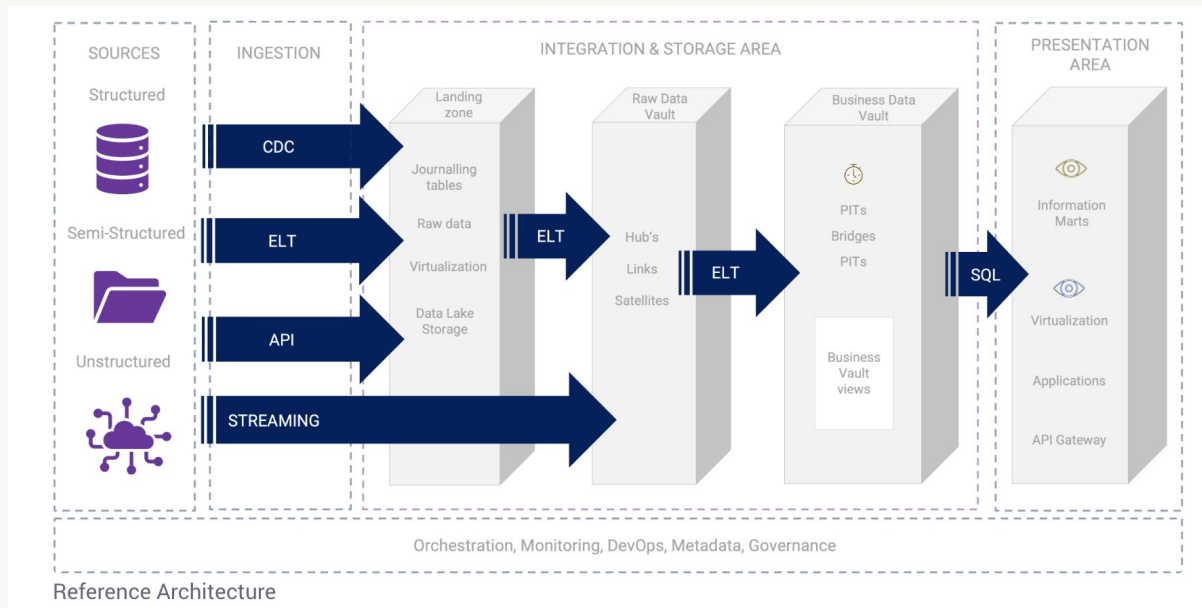
VaultSpeed helps you to model the Data Vault and delivers the data structures and the ETL needed for data loading.

The tool blends both data-driven and model-driven approaches:

- ingest metadata from the source to speed up the modeling process

- incorporate the business model to build a Data Vault model that resembles your business

VaultSpeed's data architecture is very closely related to the bronze-silver-gold setup proposed by Databricks.

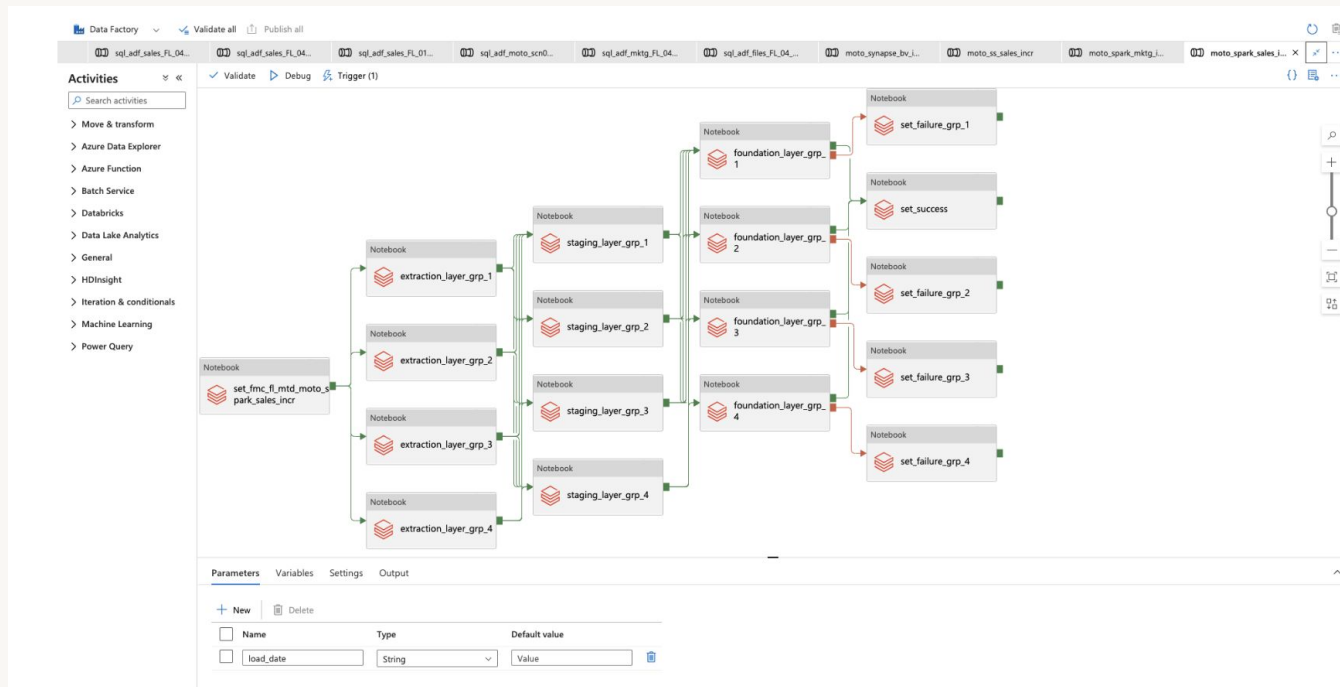


<https://vaultspeed.com/supported-technologies/databricks>



VaultSpeed Workflows

VaultSpeed generated the DDLs and ETL to automate your DataVault load in Databricks and **auto-generates** Workflows in Azure Data Factory (or Airflow..)

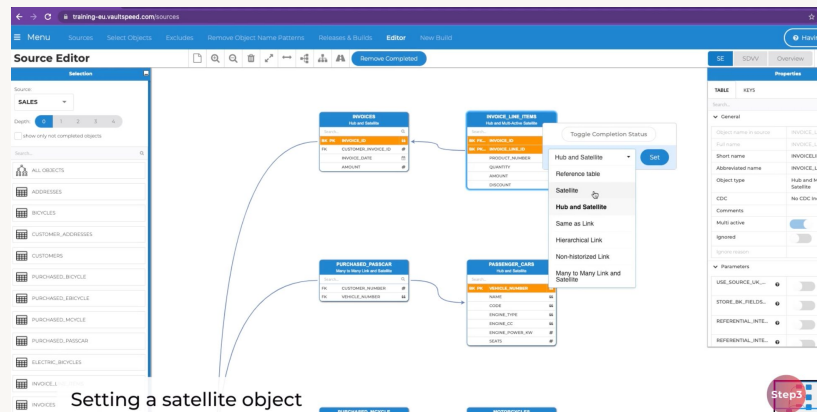


VaultSpeed's Streaming add-on module enables you to stream data into your Data Vault using Spark Structured Streaming.



Streaming Data Vault..

1. Harvest Metadata
2. Map Sources to Hubs, Links & Satellite (Modeling setup)
3. Set up the DV Data Models
4. Auto-generate DDLs and ETL code
5. Deploy Pipelines via ADF/Airflow that calls Databricks notebooks and Streaming jobs



Final thoughts..

This Lakehouse data organization approach is meant to break data silos, bring teams together, and empower them to do ETL, streaming, and BI and AI on one platform with proper governance.

Central data teams should be the enablers of innovation in the organization, speeding up the onboarding of new self-service users, as well as the development of many data projects in parallel — rather than letting the data modeling process becoming the bottleneck.

Focus on creating good Data Lakehouse standards – create Onboarding Guides for your Data and Business teams – empower self-service analytics